

 VIT Vellore Institute of Technology (Deemed to be University under section 3 of UGC Act, 1956)	Document No.	02-IPR-R003
	Issue No/Date	1/14.09.2026
	Amd. No/Date	0/00.00.0000

1. Title of the invention:

A System and Method for Schema-Free HTTP Ingest Gating Using Value-Independent Structural Fingerprints, Learned Field-Volatility Masks, and Replay-Locked Identity Protection

2. Field /Area of invention:

The invention pertains to the fields of:

- HTTP ingest middleware and pre-application request handling
- Schema-free structural fingerprinting of JSON payloads (value-independent skeletons)
- Online field-volatility classification over a bounded observation window
- Pre-store write gating of unschematized HTTP payloads, whereby an application store is opened or is not opened
- Multi-tenant ingest-state isolation, including statistics-only sharing and append-only identity protection

The invention is **not** a method of doing business, a payment scheme, or a rule that “duplicate transactions must not be charged twice.” Webhook retry handling is the **field of use**. The invention is an **ingest write-barrier** in a computer system.

3. Prior Patents and Publications from literature (provide a table summarizing the prior art)

3.1 Prior art summary table

#	Reference	Year	What it does	Key limitation (gap this invention addresses)
1	Hookdeck — Deduplication (product docs)	—	Exact body hash, or include/exclude named JSON paths , time window	Human-authored field list per provider; silent failure when the payload shape drifts
2	FastHook — <i>Webhook Idempotency And Deduplication Patterns</i>	—	include_fields / exclude_fields or full-body hash	Same: named paths or whole-body hash
3	Hookbase — <i>Receive / Deduplication</i>	—	Provider table (e.g. Stripe id, Shopify header), payload hash, or JSONata	Requires schema or provider event-id knowledge
4	Hookstream — Deduplication guide; Svix — <i>What is webhook deduplication?</i> ; Standard Webhooks specification (webhook-id)	—	Header, named body field, sender webhook-id, or payload hash	Named field or sender-supplied id; not schema-free
5	Practitioner guides (prefer event.id or chosen stable fields)	—	Hash operator-chosen identity fields; avoid full body	Human choice of identity; not learned from change frequency

6	US11595299B2 (pub. US20220038367A1) — <i>System and method of suppressing inbound payload to an integration flow of an orchestration based application integration</i> (Oracle International Corp.)	2023	Suppress unused XML/JSON XPath in an integration flow after scanning the flow	Static include/exclude from a known orchestration, not ingest retry identity
7	US12061608B2 — <i>Duplicate detection and replay to ensure exactly-once delivery in a streaming pipeline</i> (Yahoo Assets LLC)	2024	Stream-processor dedupe on an extracted event id / header concat	Needs a known key field
8	US11431662B2 — <i>Techniques for message deduplication</i> (Meta Platforms, Inc.)	2022	Message dedupe tokens + Bloom filter + timestamp window	Token is supplied, not learned from JSON shape
9	US12260270B2 — <i>Idempotent event handling</i> (SAP SE)	2025	Idempotent event handling via partitioned event-status cache	Assumes event identity already exists
10	US11449490B1 — <i>Idempotent transaction requests</i> (Amazon Technologies, Inc.)	2022	Idempotent DB transactions keyed by a client token	Sender cooperation (Idempotency-Key style)
11	US11461185B2 — <i>Event processing</i> (Capital One Services, LLC)	2022	Duplicate events by event identifiers (optional ML)	Identity given or ML-named; not volatility-ratio on a skeleton
12	US11151097B2 — <i>Dynamic schema inference and enforcement</i> (Microsoft Technology Licensing, LLC); US10803029B2 — <i>Generating JavaScript Object Notation (JSON) schema from JSON payloads</i> (Salesforce, Inc.)	2021 / 2020	Dynamic JSON schema inference for storage or documentation	Schema typing, not an ingest FORWARD/SUPPRESS mask
13	IETF draft-ietf-httpapi-idempotency-key-header — <i>The Idempotency-Key HTTP Header Field</i> ; IETF draft-mayankpanke-event-delivery-semantics — <i>Event and Webhook Delivery Semantics</i> ; Stripe-style Idempotency-Key; CDC VOLATILE columns (Snowflake / Debezium)	—	Sender-supplied key, or operator-named columns	Not schema-free HTTP ingest middleware
14	US11693849B2 — <i>Consistent structured data hash value generation across formats and platforms</i> (Dell Products LP)	2023	Canonical hash of structured data so the same record hashes the same across formats	Value-inclusive structure hash; not a learned omit set or ingest write-barrier
15	Elasticsearch <i>Fingerprint processor</i> ; OpenSearch <i>Fingerprint processor</i>	—	Hash of operator-named fields or exclude_fields for document identity	Named field list; log/document ingest, not a schema-free HTTP write-barrier

3.2 Gap analysis

Existing approach	This invention
Human-authored include/exclude JSON paths	Keep/omit paths are learned from bounded-window value-change frequency on a value-independent skeleton; no field-name list
Per-provider event-id or webhook-id table	No known identity field and no per-provider adapter
Hash of the entire JSON body	Transport-shaped values and highest raw-VR leaves are omitted only while a discriminating remainder remains
Canonical / fingerprint hash of named or all fields (Dell US11693849B2; Elasticsearch / OpenSearch)	Same: operator-named include/exclude or a value-inclusive structure hash, not a learned omit set

Fingerprint)	
Client-supplied Idempotency-Key	Gate works when the sender supplies no stable key
Treat UUID/hex tokens as automatic nonces	UUIDs and hex strings are not dropped as nonces; identity is not inferred from token shape
Copy raw payloads, or copy “this path is identity,” across tenants	Seed is statistics only ; raw values are not pooled; immunity is never inherited
Mutable flags that can unlock a learned identity path	Append-only protect set; difference-isolated replay is the only writer
Learned or static mask that retains only in-window constants	Remainder-safe fallback digest; high-VR leaves drop only while a discriminator remains

Source-code clone detection and git blame are a different problem and are not claimed.

The present system is not “Hookdeck exclude_fields plus a TTL.” The exclude set is learned from value-change frequency on a value-independent skeleton, with no field-name list.

4.Summary and background of the invention (Address the gap / Novelty)

Patentable Subject Matter Statement

The present invention is **not** a mathematical method, **not** a business method, and **not** a computer programme per se within the meaning of **Section 3(k)** of the Patents Act, 1970 (read with the CRI Guidelines). It is **not** a method of doing business and is **not** claimed as “do not process the same order twice” or “do not charge twice.”

It is a **technical system** comprising a processor coupled to a memory, **operatively coupled to an application store**, that performs **HTTP ingest write isolation**. A gate sitting in front of that store issues FORWARD (the store **may** be opened for a write) or SUPPRESS (the store is **not** opened). In the laboratory embodiment, the stand-in store (src/downstream_sink.py) is written only on FORWARD and on false-positive replay; SUPPRESS calls skip() and never writes. /stats.downstream counts writes versus writes_avoided. That countable effect — **a class of HTTP deliveries never executing a store write** — is the technical contribution.

Volatility ratios, magnitude splits, and related calculations are **not** the product of the invention. They are **embedded control parameters of the gate** (which JSON paths enter a digest that keys the write decision). They serve a larger technical process: controlling whether the application store is opened, without a provider schema and without a human-authored field list. The same interceptor addresses changing payload shapes by a value-independent skeleton rather than N filter tables per provider, which is an improvement in the functioning of ingest machinery, not a business rule.

Webhook or payment retry handling is only a **field of use**. The invention is the write-barrier.

Background

HTTP webhook ingest software must decide, **before** the application store is opened, whether a JSON body is a new event or a retry of an event already admitted. Providers change payload shape without notice. Field names are untrusted. Commercial tools require a human to configure static include/exclude lists or a provider event-id table. That configuration **rots** when the shape changes. Senders often do not supply a stable Idempotency-Key. Exact hashing of the whole body fails when timestamps, delivery ids, or attempt counters change.

Tenants of the same provider **cannot** share raw payloads. A cold tenant cannot be helped by copying another tenant’s JSON. A wrong SUPPRESS **destroys data the application never sees**, so a correction path must re-admit the event and narrow future behaviour **without unlocking the whole learned mask**. A learned mask can degenerate: if every retained field is constant in-window, distinct events would collide. These are properties of **ingest machinery**, not of accounting.

The gap

- Named include/exclude lists need a schema and fail silently on drift.
- Event-id tables need per-provider knowledge.
- Whole-body hashes treat transport noise as identity.
- UUID/hex “nonce” heuristics destroy business identifiers.
- Sharing raw events across tenants violates isolation; sharing “this field is immune” lets a donor unlock another tenant’s protect set.
- Empty or constant-only masks collide distinct events unless a remainder-safe fallback exists.

Novelty

The present invention does not track events by a named field, a provider event-id, or a hash of the whole JSON body. It learns which paths enter a tenant-scoped ingest digest from observed change frequency on a value-independent skeleton, and that digest keys a write-barrier in front of an application store. The novelty is this **combination**, not a single hash and not any one of the following steps. The system:

1. Computes a value-independent skeleton of JSON path topology ($[i] \rightarrow []$).
2. Updates a bounded-window volatility ratio (changed / total) per (tenant, skeleton, path), pairing on path intersection.
3. Merges a frequency tier and a behavioural tier (clock, monotonic/counter, magnitude) bidirectionally, each able to correct the other.
4. May seed a cold tenant with statistics only (provider:skeleton); recent_values are stripped; protected-identity immunity is never inherited.
5. Writes an append-only protect set only by difference-isolated replay.
6. If the learned mask is empty or constant-only, falls back to a remainder-safe digest, omitting highest raw-VR leaves only while a discriminator remains.
7. Provisionally omits structurally first-seen leaves from that fallback only while a learned path remains.

The laboratory embodiment does **not** include field-name heuristics, secondary-key / 2^k hedging, or a distributed / Redis store. DecisionRecord is emitted per delivery so the write decision is inspectable; it is not a separate novel mechanism.

Synergistic Technical Effects (Non-Obviousness Defense)

The combination of elements in the present invention produces synergistic technical effects that would not be predicted by combining a named include/exclude list with a whole-body hash or an Idempotency-Key. Each effect below is a property of the **combination**. No single module is the invention.

Effect 1: A value-independent skeleton plus observed change frequency produces an exclude set without a field-name dictionary.

Commercial ingest tools require a human to name include/exclude paths, or they hash the whole body. Combining those two prior methods still requires the named list: the hash does not tell the operator *which* paths to drop, and the named list does not survive a silent shape change. The present system does not take that list as an input.

Path topology (values ignored, $[i] \rightarrow []$) is the pairing key: two JSON bodies that differ only in values share a skeleton, so they can be compared. Bounded-window volatility ratio (changed / total, on the **intersection** of paths) then decides which paths enter the ingest digest. A second behavioural tier (clock correlation, monotonic/counter, magnitude) may correct the frequency tier, and the frequency tier may correct it; neither tier is used alone as a field-name guess. The exclude set is therefore an **output** of pairing on a skeleton. That is not predicted by “hash plus Hookdeck exclude_fields.” The write-barrier is keyed by that learned digest: FORWARD may open the application store; SUPPRESS does not.

Effect 2: Classification statistics may be pooled; protected-identity immunity cannot.

The obvious multi-tenant shortcut is to copy another tenant's payloads, schema, or "this path is identity" flags so a cold tenant starts faster. Copying payloads breaks isolation. Copying immunity lets a donor unlock another tenant's protect set: a path marked identity for tenant A would then be treated as identity for tenant B without tenant B ever having admitted that event.

The present system pools **statistics only** (provider:skeleton when the provider is known): STABLE / VOLATILE / UNCERTAIN tiers and observation counts, never raw field values (recent_values stripped). Seed **cannot** set is_protected_identity. The protect set is append-only, re-applied on every volatility-record read, and written **only** by difference-isolated replay of a suppressed payload against a forwarded payload. A wrong SUPPRESS can therefore be corrected by locking the divergent non-transport paths **without** unlocking the rest of the learned mask. That split — faster cold start without granting durable identity — is a data-structure constraint, not "share the idempotency key."

Effect 3: Highest raw-VR leaves are omitted only while a discriminator remains.

The obvious reaction to transport noise is to strip timestamps, or to drop every high-change path. Either rule alone fails. Keeping delivery timestamps, attempt counters, and opaque delivery strings in the digest makes a genuine retry look new (false FORWARD, store opened twice). Dropping every high-change path, or hashing only in-window constants, makes two distinct events look the same (false SUPPRESS, store never opened for the second event).

The present system omits highest raw-volatility leaves **only while a discriminating remainder stays**. In SELECTIVE mode the remainder must be mixed, not high-VR, and not a low-magnitude numeric. On the collapsed (transport-stripped) fallback, omit proceeds only while any discriminator remains. If the learned mask is empty or constant-only, that remainder-safe fallback is used instead of colliding distinct events. Structurally first-seen leaves (no record on this shape and absent from the tenant's known-path base) are provisionally omitted from the fallback only while a learned path remains, so a retry that bolts on an unlearned wrapper key does not automatically become a new digest. One remainder rule blocks both failure modes at once. That coupling is not predicted by a static exclude list plus a TTL.

Volatility ratios, magnitude splits, and related calculations remain **control parameters of the gate**, not the product of the invention. The product is whether the application store is opened.

Definitions

The following terms are used throughout this specification and claims with the specific meanings defined below.

"Application store" means a memory or storage operatively coupled to the ingest system, including but not limited to a database, queue, or laboratory stand-in store, into which an admitted HTTP JSON payload may be written. In the laboratory embodiment the stand-in store is src/downstream_sink.py.

"Write-barrier" (also **"gate"**) means the decision point sitting in front of the application store that issues FORWARD or SUPPRESS before a store write is executed. Volatility ratios, magnitude splits, and related calculations are control parameters of the write-barrier, not the product of the invention.

"FORWARD" means the gate admits the HTTP delivery and the application store **may** be opened for a write.

"SUPPRESS" means the gate treats the HTTP delivery as a retry of an event already admitted, the application store is **not** opened, and the payload may be placed in the shadow buffer.

"Tenant identifier" means an identifier that scopes ingest state (volatility records, digest keys, protect set, seed application) so that one tenant's payloads and immunity are not used as another tenant's.

"Provider identifier" means an optional label of the HTTP source (for example a webhook provider name). When known and not unknown, it may form part of a statistics-only seed pool key together with a skeleton. It is not a field-name list and not a per-provider event-id table.

"Key path" means a dot-delimited location of a leaf in a JSON object, with array indices normalized to a common placeholder [] (laboratory form items[].id).

"Value-independent skeleton" (also **"skeleton hash"**) means a digest of JSON **path topology only**, independent of field values and independent of field-name semantics. Two payloads that share the same key-path structure share a skeleton even when values differ. Array indices are normalized to []. Values are not hashed into the skeleton.

"Pairing window" means a bounded time interval within which same-skeleton payloads for the same tenant may be compared. The laboratory default is 300 seconds.

"Path intersection" means that volatility observations are updated only on key paths present in both paired payloads, so that a change in array length does not invent observations for missing leaves.

"Volatility ratio" or **"VR"** means, for a key path, changed observations divided by total observations in a bounded deque. The laboratory default window is 50. A path recorded as protected identity reports VR 0.

"STABLE", **"VOLATILE"**, and **"UNCERTAIN"** mean the first-tier (frequency) labels of a path: VR below the stable threshold; VR above the volatile threshold; or between the thresholds or below the minimum pair-observation count. Laboratory defaults: stable threshold 0.05, volatile threshold 0.70. These thresholds are embedded control parameters of the gate, not the claimed product.

"Bidirectional two-tier merge" means combining the frequency tier with a second behavioural tier (clock correlation, monotonic or counter behaviour, and value magnitude) so that either tier may correct the other. A path in the protect set is forced STABLE.

"Identity magnitude threshold" means the design-default numeric split (laboratory 50.0) used only in the behavioural tier to separate low-magnitude counters from high-magnitude sequences. It is not a re-swept empirical claim and is not used as a keep-set identity test.

"Ingest digest" (also **"mask"**, **"mask hash"**) means the hash of selected path values that keys the tenant-scoped write-barrier: whether the application store may be opened.

"SELECTIVE" means an ingest digest over learned non-VOLATILE paths, after optional VR-gated omit.

"Value-shape transport" means a value treated as delivery/transport noise rather than event identity: an ISO 8601 or RFC3339 string, or an epoch in seconds, milliseconds, or microseconds within two years of the webhook's observation time (`current_time`). UUIDs and hex tokens are **not** automatic nonces and are not value-shape transport solely by token shape.

"Non-discriminating mask" means a learned SELECTIVE digest that is empty or whose retained values are constant in-window, so that distinct events would collide if that digest were used as the sole gate key.

"TRANSPORT_STRIPPED" (also **"collapsed fallback"**) means the remainder-safe fallback digest: payload minus value-shape transport, then VR-gated omit, then first-seen probation, still retaining a discriminator.

"VR-gated omit" means omitting highest raw-volatility leaves from a SELECTIVE digest or a collapsed digest **only while a discriminating remainder remains**. In SELECTIVE mode the remainder must be mixed, not high-VR, and not a low-magnitude numeric. On the collapsed fallback, omit proceeds only while any discriminator remains. Protected paths are not omit candidates.

"Discriminating remainder" (also **"discriminator"**) means at least one retained path-value in the digest that can still differ across distinct events, so that omitting transport or high-VR leaves does not collapse two identities onto one key.

"First-seen probation" means provisionally omitting, from the fallback digest only, a leaf that has no volatility record on the current skeleton **and** is absent from the tenant's known-path base, and doing so only while at least one previously learned leaf remains.

"Known-path base" means the set of key paths previously observed for that tenant, used as the probation baseline (not a field-name dictionary).

"Tenant-scoped suppression store" means the store of recently seen ingest-digest keys, keyed by tenant identifier and digest, against which the gate queries before opening the application store.

"Keep-set transition match" means, after a digest miss, re-applying this delivery's **nested** digest keep-paths to a previously admitted payload of the same tenant that is path-set compatible. It is the same gate, not a new classifier, and does not use the identity magnitude threshold. Top-level keep-paths that caused the digest miss are not re-applied.

"Shadow buffer" means a time-bounded hold of a SUPPRESS payload so that a false-positive report may re-admit it. SUPPRESS does not write the application store.

"Protected identity path" means a key path in the append-only protect set for (tenant, skeleton). Replay is the only writer. The set is re-applied on every volatility-record read. Protected paths are forced STABLE and are not omit candidates.

"Difference-isolated replay" means, on a false-positive report, comparing a suppressed payload to a forwarded payload, protecting only divergent non-transport paths, and optionally re-admitting the held payload as a store write, without unlocking the rest of the learned mask.

"Statistics-only seed" means pooling classification tiers and observation counts across tenants that share a provider identifier and a skeleton, with raw payload values stripped, under-observed fields skipped, and **no** write into any tenant's protect set. Protected-identity immunity is never inherited.

"DecisionRecord" means a structured account of one FORWARD or SUPPRESS decision: digest mode,

digest keep-paths, omitted paths, protected paths, probationary paths, second-tier corrections, and whether a seed was applied. It is inspectability, not a separate novel mechanism.

"**Field-name list**" (also "**field-name dictionary**") means a human-authored or hardcoded set of JSON names or paths used as identity or exclude rules (for example id, timestamp). The laboratory embodiment does not use a field-name list. Schema-blindness is enforced by tools/check_no_fieldname_heuristics.py.

5.Objective(s) of Invention

1. To provide a computer-implemented ingest write-barrier that decides FORWARD vs SUPPRESS without a provider schema or field-name list, such that SUPPRESS does not open the application store.
2. To fingerprint payload structure independently of values so same-shape deliveries can be paired.
3. To classify paths by bounded-window change frequency, refined by a second behavioural tier, with either tier able to correct the other.
4. To hash a learned subset of paths so retries that differ only in transport or high-volatility delivery leaves can share a gate key, while distinct identities still differ.
5. To fall back, when the learned mask is empty or constant-only, to a remainder-safe transport-stripped digest.
6. To provisionally omit structurally novel leaves from that fallback only while a learned discriminator remains.
7. To pool statistics only across tenants of the same provider and skeleton, without inheriting immunity.
8. To lock identity paths only via difference-isolated replay into an append-only store.
9. To emit a DecisionRecord so each write-barrier decision is inspectable.

6.Working principle of the invention (in brief)

The system operates as a seven-stage HTTP ingest pipeline **in front of** the application store. Webhook retry handling is the field of use. The invention is the write-barrier: **FORWARD** may open the store; **SUPPRESS** does not.

Stage 1: INGEST (M1). A JSON body arrives with a tenant identifier and an optional provider identifier. A lock per (tenant, skeleton) serialises work on payloads of the same shape.

Stage 2: SKELETON (M2). Path topology is extracted and hashed. Values are ignored. Array indices [0], [1], ... are written as [] on the skeleton, so items[0].sku and items[1].sku share one skeleton path. Pairing and VR still use the indexed form. Two bodies that differ only in values share a skeleton and can be paired. No field-name list is consulted.

Stage 3: PAIR AND SEED (M3, M10). Same-skeleton payloads for that tenant within the pairing window are compared. Volatility ratio is updated on the **intersection** of paths, so a change in array length does not invent observations for missing leaves. If local VR is empty, a statistics-only seed (provider:skeleton when the provider is known) may fill STABLE / VOLATILE / UNCERTAIN tiers. recent_values are not pooled. The seed **cannot** set is_protected_identity.

Stage 4: CLASSIFY (M4, M9). Tier 1 maps VR to STABLE, VOLATILE, or UNCERTAIN (laboratory thresholds 0.05 / 0.70; minimum pair observations). UNCERTAIN stays in the SELECTIVE digest until proven VOLATILE. Tier 2 detects clock-shaped values, counters, monotonic sequences, and magnitude. Identity magnitude threshold **50.0** is used in Tier 2 and in mask remainder / collapse tests so a 0/1 retry counter is not treated as a discriminator; it is **not** a keep-set identity test. Merge is **bidirectional**: either tier may correct the other. Protected paths are forced STABLE. The laboratory merge rules are tabulated in the Description.

Stage 5: MASK (M5). Non-VOLATILE paths form a SELECTIVE ingest digest, then VR-gated omit if a discriminating remainder remains (SELECTIVE remainder must be mixed, not high-VR, and not a low-magnitude numeric). If the learned mask is empty or constant-only, a TRANSPORT_STRIPPED digest is built instead: drop value-shape transport, VR-gated omit while any discriminator remains, then first-seen

probation. UUIDs and hex tokens are not treated as automatic nonces.

Stage 6: GATE (M6). The digest is looked up in the tenant-scoped suppression store. Three outcomes, all ending at the store: key found within the suppression window → **SUPPRESS** (store not opened); miss, then nested keep-set match → **SUPPRESS** (store not opened); no key and no keep-set match → **FORWARD**, the key is inserted, and the application store **may** be opened. Keep-set re-applies this delivery's **nested** digest keep-paths only (same gate, not a new classifier). The gate table is in the Description.

Stage 7: SHADOW AND REPLAY (M7, M8). On SUPPRESS the payload is held in a time-bounded shadow buffer; the store is not written. A false-positive report compares suppressed vs forwarded, appends only divergent non-transport paths to the protect set, and may re-admit the held payload as a store write, without unlocking the rest of the learned mask.

After a local (non-seeded) classify, eligible VR records may be registered into the statistics-only pool. That registration never writes the protect set. Each decision may emit a DecisionRecord (inspectability, not a separate stage). The numerical thresholds and windows used in this laboratory embodiment are design defaults, not the claimed product; they are recited in the Description.

7. Description of the invention in detail (Include drawing and or photograph as needed)

7.0 Laboratory embodiment

This section describes the **laboratory embodiment** that exists in live src/ and is exposed by FastAPI app.py. It is **TRL-4**: the modules and the HTTP gate are validated in a laboratory, in memory, on a single node. It is not a distributed production gate, not a Redis-backed store, and not a claim of a production SLO. The HTTP surface (JSON POST, typically a webhook) is the **field of use**. The invention is an **ingest write-barrier**: on FORWARD the application store **may** be opened; on SUPPRESS it is **not** opened. Volatility ratios, magnitude splits, and related calculations are **control parameters of that gate**, not the product of the invention.

In the laboratory, src/downstream_sink.py stands in for the application store. app.py writes that sink on FORWARD and on false-positive replay, and calls skip() on SUPPRESS. /stats.downstream counts writes versus writes_avoided. That countable effect — a class of HTTP deliveries never executing a store write — is the technical contribution.

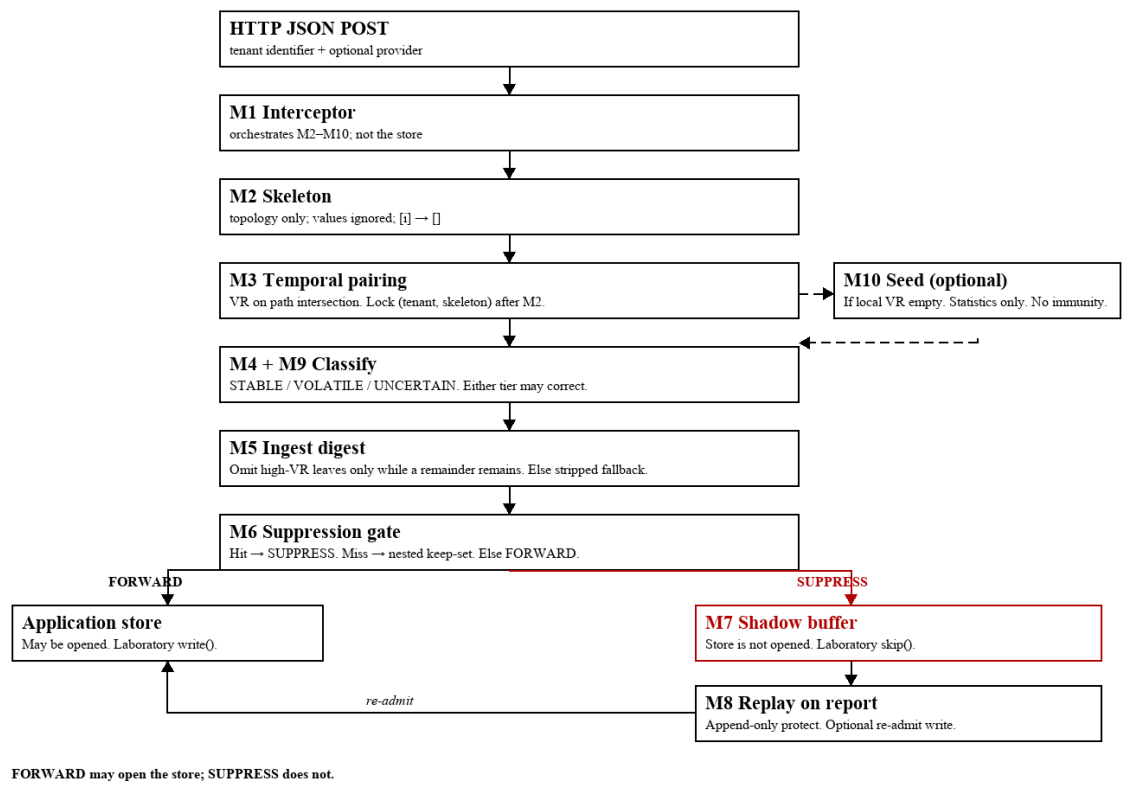
No field-name dictionary is used. Schema-blindness is enforced by tools/check_no_fieldname_heuristics.py. Secondary-key / 2^k hedging and a distributed store are **not** in this embodiment.

Laboratory defaults (not the claimed product). Dataclass SystemConfig uses min_pair_observations=3; the laboratory server app.py starts the engine with 2. Thresholds **0.05 / 0.70 / 50.0** are design defaults of this embodiment. **50.0** is the identity-magnitude split: it is used in Tier 2 (low-magnitude counter versus high-magnitude sequence) and again in mask collapse / SELECTIVE remainder tests so a 0/1 retry counter is not treated as a discriminator. It is not a keep-set identity test.

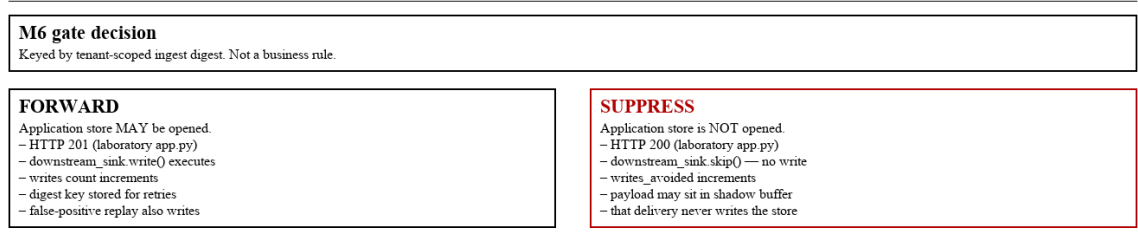
Figures 1–6 below are engineering drawings of this embodiment. Values in worked examples are illustrative (Stripe-like shape). They are not a provider adapter and not a field-name list.

7.1 Figure 1 — Apparatus: ingest write-barrier in front of the store

The pipeline sits **in front of** the application store. M1–M10 compute a tenant-scoped ingest digest and a FORWARD / SUPPRESS decision. Only FORWARD (and later, false-positive replay) may open the store. SUPPRESS holds the payload in a shadow buffer and does not write.

FIG. 1 *HTTP ingest write-barrier**Figure 1. Laboratory ingest pipeline (M1–M10) in front of the application store.***Figure 1. HTTP ingest write-barrier (M1–M10 + store).**

The combination that is novel is not “hash JSON” and not any single module. It is: a value-independent skeleton as the pairing key; bounded-window VR on path intersection; bidirectional two-tier merge; a remainder-safe learned digest (VR-gated omit only while a discriminator remains, with transport-stripped fallback and first-seen probation); a tenant-scoped gate that may re-apply nested keep-paths after a digest miss; statistics-only seed that cannot grant immunity; and an append-only protect set writable only by difference-isolated replay — all ending in whether the store is opened.

7.2 Figure 2 — Technical effect: store opened versus not opened**FIG. 2** *Store opened versus not opened**Figure 2. A SUPPRESS delivery never executes a store write.***Figure 2. FORWARD may write; SUPPRESS does not open the store.**

Laboratory HTTP mapping (app.py): FORWARD → HTTP 201 and downstream_sink.write(); SUPPRESS → HTTP 200 and downstream_sink.skip(). The shadow buffer is not the application store. Holding a suppressed payload is not a store write.

7.3 Figure 6 — Value-independent skeleton (pairing key, not a schema)

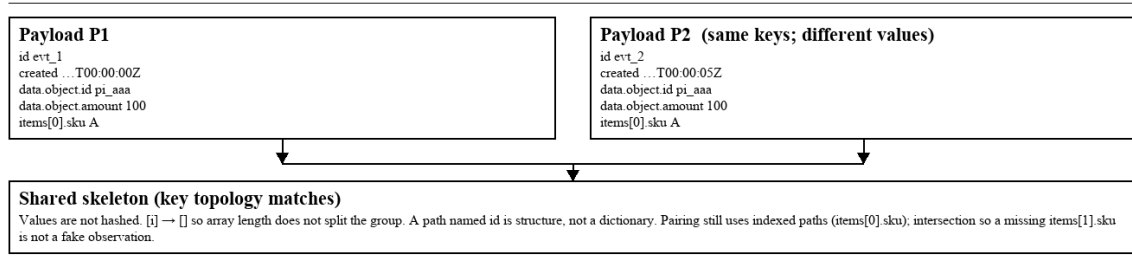
FIG. 6 Value-independent skeleton

Figure 6. Two bodies that differ only in values share a skeleton. No field-name list.

Figure 6. Two bodies that differ only in values share a skeleton and can be paired.

extract_key_paths (src/skeleton_hash.py) walks the JSON object and emits dot paths. Array elements are first recorded with indices (items[0].sku). compute_skeleton_hash then replaces [i] with [], sorts, deduplicates, and SHA-256-truncates (laboratory 16 hex characters). Values are not hashed into the skeleton. Two bodies that share key topology therefore share a skeleton even when id, timestamps, and amounts differ, and can be paired.

Two particular facts follow, and both are required for enablement:

1. **Field names are structure, not a semantic dictionary.** A path named id is not treated as identity because it is named id. Schema-blindness is the point: the exclude set is learned from change frequency, not from a list of names.
2. **Skeleton hash and VR records are not the same string form.** The skeleton hash is length-independent (items[].sku) so a longer array does not split the pairing group. Pairing, VR, and the digest still use indexed paths (items[0].sku). Updates use **path intersection**: if a retry has items[0].sku but not items[1].sku, the missing leaf is not given a fake observation.

A newly added **top-level** object key changes the skeleton. That delivery is not automatically a new business event: first-seen probation (fallback digest) and keep-set transition match (nested digest keep-paths after a miss) are the later couplings that stop a bolted-on wrapper from forcing a false FORWARD, without turning every first-ever shape into a collision.

Illustrative paths (values are examples, not a field-name list):

Payload path	Skeleton path	Treatment
customer.id	customer.id	Kept as written
items[0].sku	items[].sku	Array index normalised
items[1].sku	items[].sku	Same skeleton path as items[0].sku
data.object.amount = 4200	data.object.amount	Value ignored

The skeleton hash uses the middle column. Pairing, VR, and the ingest digest still use indexed paths (items[0].sku). Path intersection so a missing items[1].sku is not given a fake observation.

7.4 Module 1 — HTTP interceptor (src/interceptor.py)

WebhookDeduplicationEngine.process_webhook is the orchestrator. It binds M2–M10 and returns a SuppressionResult plus a DecisionRecord.

Locking. The skeleton is computed **before** the lock. Same-shape work is then serialised by _get_lock(tenant_id, skeleton_hash). _total_processed is incremented under _engine_lock after the shape lock is released, so different skeletons may run concurrently. The lock map is **not shrunk** in the laboratory embodiment. That is a disclosed memory limit, not a claimed production property.

Order under the shape lock (this is the working order):

1. M3 pair / observe.
2. Read volatility records (_apply_protected on every read).

3. If local VR is empty and seed is enabled, M10 may inject statistics-only records (just_seeded).
4. M4 Tier 1 classify.
5. If enable_bidirectional_merge, M9 apply_overrides; merged tiers are written back onto the volatility records.
6. M5 build_mask (always the normal builder; seed does not call build_mask_with_explicit_fields).
7. M6 evaluate.
8. Attach DecisionRecord (digest mode, keep-paths **after** omit, omitted paths, protected paths, probationary paths, Tier-2 corrections, seed_applied).
9. On SUPPRESS, M7 shadow store.
10. If not just_seeded, register eligible VR into the pool (M10). Protected-identity records may be registered as statistics; the registration **never** writes _protected_paths.

DecisionRecord is inspectability of the write decision. It is not a separate novel mechanism and not a seventh pipeline stage.

7.5 Module 3 — Temporal pairing and append-only protect set (src/temporal_pairing.py)

Same-skeleton payloads for **one tenant** inside the pairing window (lab **300 s**) are compared. The most recent in-window candidate is used. VR for each intersecting path is changed observations divided by total observations in a bounded deque (lab **50**). A path recorded as protected identity reports VR **0** and is forced STABLE.

_protected_paths is keyed by (tenant, skeleton). protect_identity_fields is the **replay-only** writer (append-only). _apply_protected runs on every volatility-record read so later drift, reclassification, and seed cannot unlock a protected path. When seed records are injected, set_volatility_records overwrites is_protected_identity from **this tenant's** protect set, not from the donor.

_tenant_known_paths is the first-seen probation baseline: every path this tenant has classified on **any** shape. It is not a field-name dictionary.

7.6 Modules 4 and 9 — Bidirectional two-tier classify (src/classifier.py, src/tier2_classifier.py)

Tier 1 (frequency). If total_observations < min_pair_observations → UNCERTAIN. Else VR < **0.05** → STABLE; VR > **0.70** → VOLATILE; otherwise UNCERTAIN. Protected identity → STABLE regardless. **UNCERTAIN stays in the SELECTIVE digest** until proven VOLATILE. That choice is particular: a cold path is not dropped as noise, and a mid-VR identity path (changes across distinct events, matches on retry) is not stripped.

Tier 2 (behaviour). Clock-shaped values (ISO / RFC3339 strings; epoch s/ms/μs within two years of injectable current_time) tend VOLATILE. Low-magnitude pure-numeric counters / monotonic sequences tend VOLATILE. High-magnitude monotonic sequences tend STABLE (so an incrementing business cursor is not treated as a delivery counter). Prefixed / opaque sequences that only look monotonic stay in-mask unless VR already proves they change on almost every pair.

Bidirectional merge (`apply\_overrides`). Either tier may correct the other. Protected paths are forced STABLE (Tier 1 and Tier 2 cannot unlock them). Transport evidence makes VOLATILE. High-magnitude monotonic makes STABLE. Low-magnitude counters make VOLATILE. Tier 1 STABLE is **not** blindly overwritten by a non-transport, non-counter Tier 2 guess. Neither tier is used as a field-name guess.

Observed behaviour	Resulting label	Note
ISO / RFC3339 string, or epoch s/ms/μs within two years of current_time	VOLATILE	Value-shape transport
Low-magnitude pure-numeric counter or monotonic sequence (0...50.0)	VOLATILE	Delivery / attempt counter; not a remainder
High-magnitude monotonic sequence	STABLE	Business cursor is not treated as a

		delivery counter
Path in the protect set	STABLE	Replay-locked identity; VR reports 0
Tier 1 STABLE and Tier 2 is non-transport, non-counter	STABLE retained	Tier 1 is not overwritten blindly
Insufficient pair observations, or VR between 0.05 and 0.70	UNCERTAIN	Stays in the SELECTIVE digest until proven VOLATILE

50.0 is this magnitude split and the SELECTIVE remainder test. It is not a keep-set identity test.

7.7 Module 5 — Learned ingest digest (src/mask_builder.py)

The digest is the hash that keys the write-barrier. Laboratory hash: canonical path=json lines, SHA-256 truncated to 32 hex characters.

SELECTIVE mode. Non-VOLATILE paths (STABLE and UNCERTAIN) are hashed. Protected paths are forced into the keep-set. Then, if enable_vr_gated_fallback, Pass B omits highest **raw** VR leaves (VR > 0.70, not protected, has observations) **only while a discriminating remainder remains**. In SELECTIVE mode that remainder must be mixed, not high-VR, and **not** a low-magnitude numeric (0...50.0). Retry counters that have seen 0 and 1 in-window are therefore not remainders. Protected paths are not omit candidates.

Non-discriminating (collapsed) learned mask. If the keep-set is empty, or every retained value is constant in-window **and** a non-transport discriminator was excluded, that learned mask is **not** used as the gate key. Distinct events would otherwise collide (false SUPPRESS, store never opened for the second event).

TRANSPORT_STRIPPED fallback. Payload minus value-shape transport, then VR-gated omit while **any** discriminator remains, then first-seen probation. Transport is value-shape plus clock correlation: ISO/RFC3339; epoch seconds, milliseconds, or microseconds within **two years** of current_time. Nanosecond epoch is omitted because 64-bit snowflake identifiers overlap that numeric range. **UUIDs and hex tokens are not automatic nonces** and are not transport solely by token shape.

First-seen probation (fallback only). A leaf with no VR record on this skeleton **and** absent from the tenant's known-path base is provisionally omitted from the fallback, **only while a previously learned leaf remains**. A first-ever shape keeps the strict digest so two distinct events cannot collide on an empty learned remainder. Two distinct events whose only difference is a first-ever leaf on a new shape can still collide; difference-isolated replay is the designed correction path. That hole is disclosed.

Seed path. build_mask_with_explicit_fields exists and is **unused** by the interceptor. Seed injects volatility records; the interceptor then calls normal build_mask. Collapse sampling, when seeded recent_values is empty, uses **this tenant's current payload**, not another tenant's values.

DecisionRecord keep/omit lists are the **digest** paths after Pass B, not a pre-omit wish list.

7.8 Figure 4 — Remainder-safe omit (why the combination is non-obvious here)

FIG. 4 Remainder-safe omit

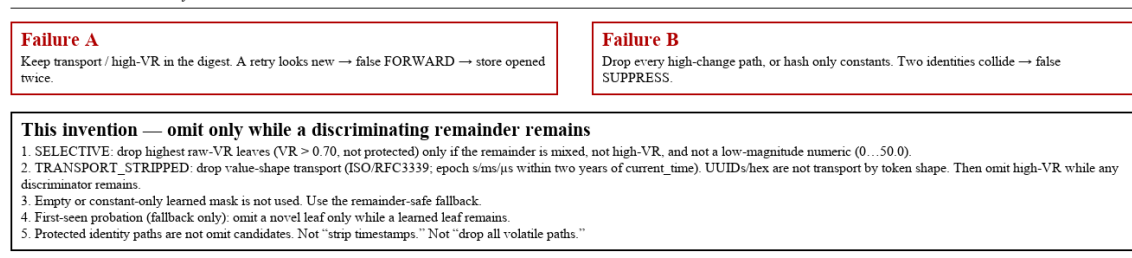


Figure 4. Highest raw-VR leaves drop only while a discriminator stays.

Figure 4. Highest raw-VR leaves drop only while a discriminator stays in the digest.

The obvious prior combination is “hash the body, or drop named timestamps.” Either rule alone fails. Keeping transport / high-VR leaves makes a genuine retry look new (false FORWARD, store opened twice). Dropping every high-change path, or hashing only in-window constants, makes two identities share

a key (false SUPPRESS). The present remainder rule is a **coupling**: omit proceeds only while a discriminator stays. That is Effect 3. It is not “strip timestamps” and not “drop all volatile paths.”

7.9 Module 6 — Suppression gate (src/suppression_gate.py)

Keys are tenant:mask_hash. There is **no** cross-tenant key fallback. Hit within the suppression window (lab **600 s**) → SUPPRESS. Mask entries expire on TTL (lab **3600 s**).

On miss, if enable_keepset_transition_match (laboratory default on), the gate re-applies this delivery’s **nested** digest keep-paths (mask.stable_fields whose path contains a .) to a previously admitted payload of **this tenant** that is path-set compatible (subset or superset — a retry may add a top-level leaf such as delivery_attempt). Nested keep values must match and must not be value-shape transport. Top-level keep-paths that caused the digest miss are **not** re-applied. A different nested identity (for example a different data.object.id) FORWARDS even if a nested customer string or amount would overlap.

Still no match → FORWARD, insert the key, remember the payload for later keep-set comparison.

This is the **same gate**, not a seventh classifier, and it does not use 50.0 as an identity test.

Condition	Gate action	Effect on the application store
Digest key found within the suppression window (lab 600 s)	SUPPRESS	Store is not opened; payload held in the shadow buffer
Digest miss, then nested keep-set match	SUPPRESS	Store is not opened; payload held in the shadow buffer
No key and no nested keep-set match	FORWARD	Store may be opened; digest key inserted

Keep-set uses nested digest keep-paths only (path contains a .). Top-level keep-paths that caused the miss are not re-applied. Flat payloads with no nested keep-paths: keep-set is a no-op; the digest alone decides (laboratory first-retry miss on integer identity is the disclosed **2.5% FNR** class). Nested keep-paths that are only shared constants can still false-SUPPRESS on some JSON shapes. Stripe-like data.object.id differing is the designed-safe FORWARD case. This description does not claim 0% FNR on every JSON shape.

7.10 Figure 5 — Keep-set transition match

FIG. 5 *Keep-set transition match*

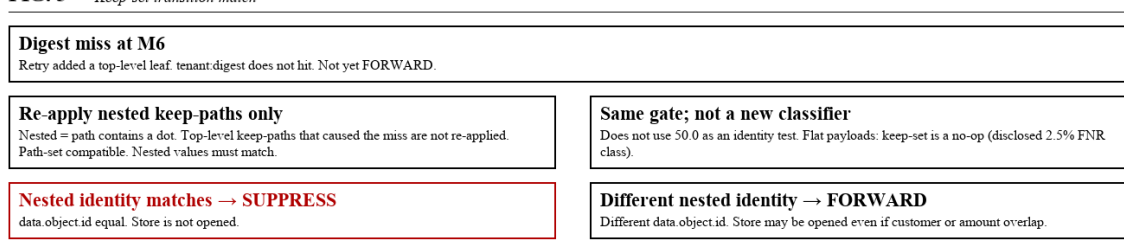


Figure 5. Nested keep-paths of this delivery are re-applied after a digest miss.

Figure 5. After a digest miss, nested keep-paths of this delivery are re-applied to a previously admitted payload of this tenant.

7.11 Module 7 — Shadow buffer (src/shadow_buffer.py)

SUPPRESS payloads are held, tenant-prefixed, for a grace period (lab **1800 s**), capacity-bounded (lab **10000**). Size eviction is global. That is not a cross-tenant data leak: another tenant cannot read the entry. The shadow is not the application store.

7.12 Module 8 — Difference-isolated replay (src/replay_feedback.py)

POST /report-false-positive is tenant-scoped (mask_hash or webhook_id). The controller retrieves the shadow entry, diffs against the forwarded payload that established the digest, resolves nested array paths, **skips** value-shape transport, and protect_identity_fields only on divergent paths. If no value difference

remains after transport skip, it falls back to non-transport digest keep-paths of the suppressed payload (still not a field-name list). Optional re-admit invokes the bound callback, which in app.py writes the application store. The digest key is then removed from the gate so the re-admitted event is not immediately re-suppressed. `current_time` is injectable so laboratory clocks and live clocks stay consistent. Replay **does not** unlock the rest of the learned mask. That is the correction path for a wrong SUPPRESS without throwing away the exclude set.

7.13 Module 10 and Figure 3 — Statistics-only seed cannot grant immunity

FIG. 3 *Statistics may be pooled; immunity cannot*

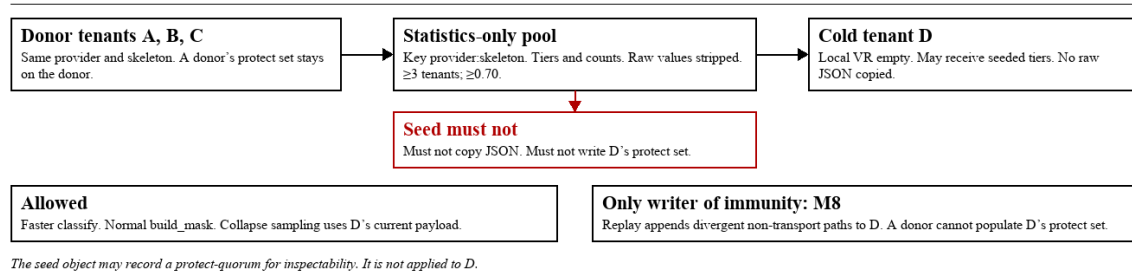


Figure 3. Cross-tenant seed is statistics-only. Immunity is never inherited.

Figure 3. Seed pools VR statistics; immunity is never inherited.

Pool key is provider:skeleton when provider is not unknown. A seed exists only with at least `min_tenants_for_seed` (lab 3) and `seed_confidence_threshold` (lab 0.70). Register skips under-observed fields (below `min_pair_observations`), except that locally protected paths may still be registered as statistics. STABLE / VOLATILE / UNCERTAIN may seed. `recent_values` are cleared at the privacy boundary.

Dual fence on immunity. The seed **object** may record a protect-quorum for inspectability (`CrossTenantSeed.protected_fields`). `seed_new_tenant` does **not** copy that quorum onto seeded records. `set_volatility_records` then forces `is_protected_identity` from the receiving tenant's own `protected_paths` only. A donor cannot populate or unlock the receiving tenant's protect set. UNCERTAIN is seedable. Identity fields that change between distinct events (VR ~0.5) must be inherited as in-mask, or a new tenant would inherit only constants plus transport and collapse to a transport-stripped digest. That is a working constraint, not a claim that seed is itself identity.

7.14 Downstream application sink (not a pipeline module)

`src/downstream_sink.py` is the laboratory stand-in for the store **behind** the gate. FORWARD and false-positive replay write; SUPPRESS skip(). This is how Independents 1(g) and 2(f) are embodied. It is not M11, not a classifier, and not a novelty bullet.

7.15 End-to-end traces (illustrative Stripe-like shape)

Values below are illustrative. The system does not look up the names `id`, `created`, or `data.object.id`.

Trace A — cold first delivery. Empty local VR, no seed. Almost every path is UNCERTAIN and therefore in SELECTIVE. Digest is new → FORWARD → store may be opened. Eligible VR is not yet registered if under-observed.

Trace B — genuine retry after learning. Same skeleton, same tenant, pairing window. ISO created / attempt counter classified VOLATILE (Tier 2 and/or VR). Nested identity remains. SELECTIVE digest matches, or digest misses and nested keep-set matches → SUPPRESS → store not opened; shadow holds the body.

Trace C — distinct nested identity. Different `data.object.id`, overlapping customer string or amount. Nested keep values differ → FORWARD → store may be opened. This is the false-SUPPRESS guard for the nested case.

Trace D — empty / constant learned mask. If every retained path is an in-window constant and a non-transport discriminator was excluded, TRANSPORT_STRIPPED is used instead of colliding distinct events.

Trace E — wrapper key on retry. New top-level leaf changes topology. Probation may omit that leaf from the fallback while a learned path remains; keep-set may still SUPPRESS on nested identity. A first-ever shape does not get that probation.

Trace F — cold tenant with seed. Three prior tenants of the same provider+skeleton, confidence ≥ 0.70 , local VR empty. Seeded tiers allow SELECTIVE without copying payloads. Protect set of the cold tenant remains empty until **that** tenant's own replay.

Trace G — false SUPPRESS report. Shadow retrieve $\rightarrow$ diff $\rightarrow$ append only divergent non-transport paths $\rightarrow$ optional store write. Subsequent deliveries keep those paths STABLE.

7.16 Scope of this embodiment

This description does not present 0% FNR on every JSON shape, a production service-level figure, TRL-5, a Redis store, a field-name dictionary, UUID-as-nonce treatment, or seed-copied immunity. Designed-mix laboratory FPR **0%** and ablation FNR **2.5%** are reported in Experimental Validation; they are not a promise of the modules above. The lock map is not shrunk. min_pair_observations is **2** on the laboratory server and **3** in the dataclass default. Nested-constants false-SUPPRESS on some JSON shapes is disclosed.

8.Experimental Validation Results

These results are laboratory evidence that the embodiment implements the write-barrier: FORWARD may open the application store; SUPPRESS does not. /stats.downstream counts writes versus writes_avoided. Schema-blindness and adaptive-versus-static drift show that the exclude set is **learned**, not a named field list. Thresholds **0.05 / 0.70 / 50.0** were held fixed for every run reported here.

py -m pytest tests/ -q and py tools/check_no_fieldname_heuristics.py were re-run on **16 September 2026** (94 passed; AST SUCCESS). Ablation, drift, adaptive-versus-static, and stress figures are laboratory records on the same fixed thresholds.

8.1 Test environment

Item	Laboratory setting
Platform	Windows, Python 3.12
Embodiment	FastAPI laboratory server (app.py); in-memory stores; single node
TRL	TRL 4 — component and system validated in a laboratory environment
Schema-blindness	py tools/check_no_fieldname_heuristics.py $\rightarrow$ SUCCESS
Fixed knobs	Stable / volatile / magnitude 0.05 / 0.70 / 50.0 held for every run below

8.2 Execution evidence

The laboratory HTTP console is <http://localhost:8000/> (py app.py). The mapping in app.py is particular: FORWARD $\rightarrow$ HTTP **201** and downstream_sink.write(); SUPPRESS $\rightarrow$ HTTP **200** and downstream_sink.skip(). OpenAPI is at /docs. py experiments/live_test.py is an optional live check against the same FastAPI app (port 8000, or in-process TestClient if the port is down).

Step	Action	Gate	Application store
1	Start py app.py	Console up at http://localhost:8000/	—
2	Preset Stripe Charge	FORWARD, HTTP 201	Opened (downstream_sink.write)
3	Preset Stripe Retry Duplicate	SUPPRESS, HTTP 200	Not opened (skip)
4	GET /stats $\rightarrow$ downstream	—	writes increments on step 2; writes_avoided increments on step 3

5	OpenAPI /docs	Interface available	—
6	Optional py experiments/live_test.py	Same gate on the FastAPI app	Same store rule

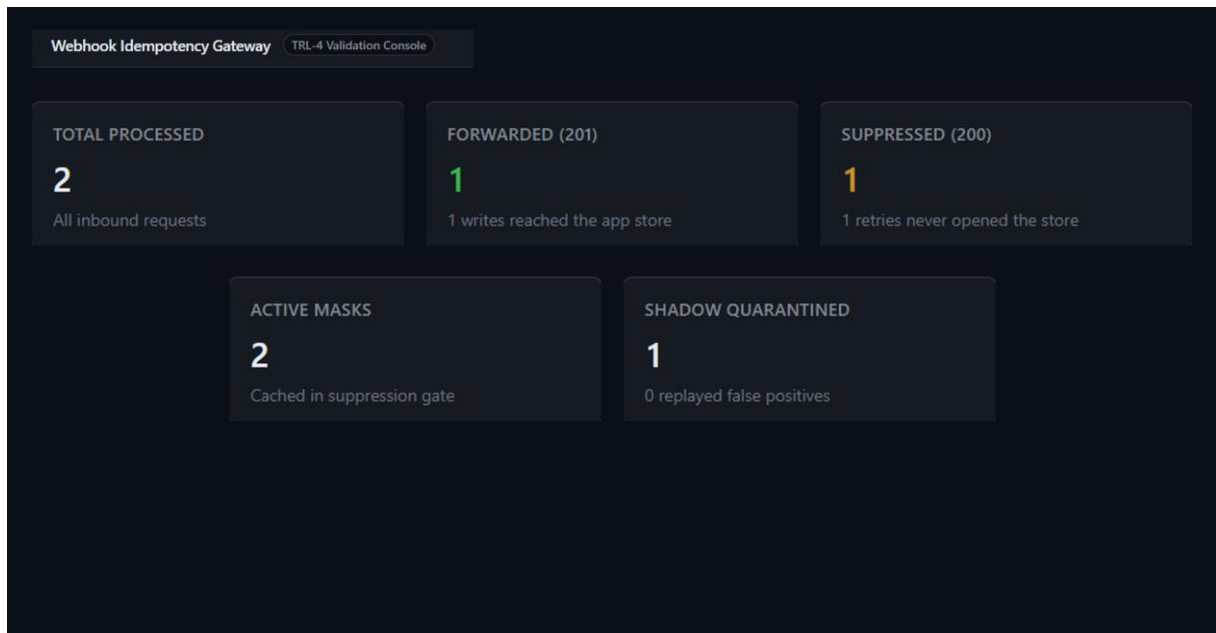


Figure 7(a). Store counters: 1 write reached the app store; 1 retry never opened the store.

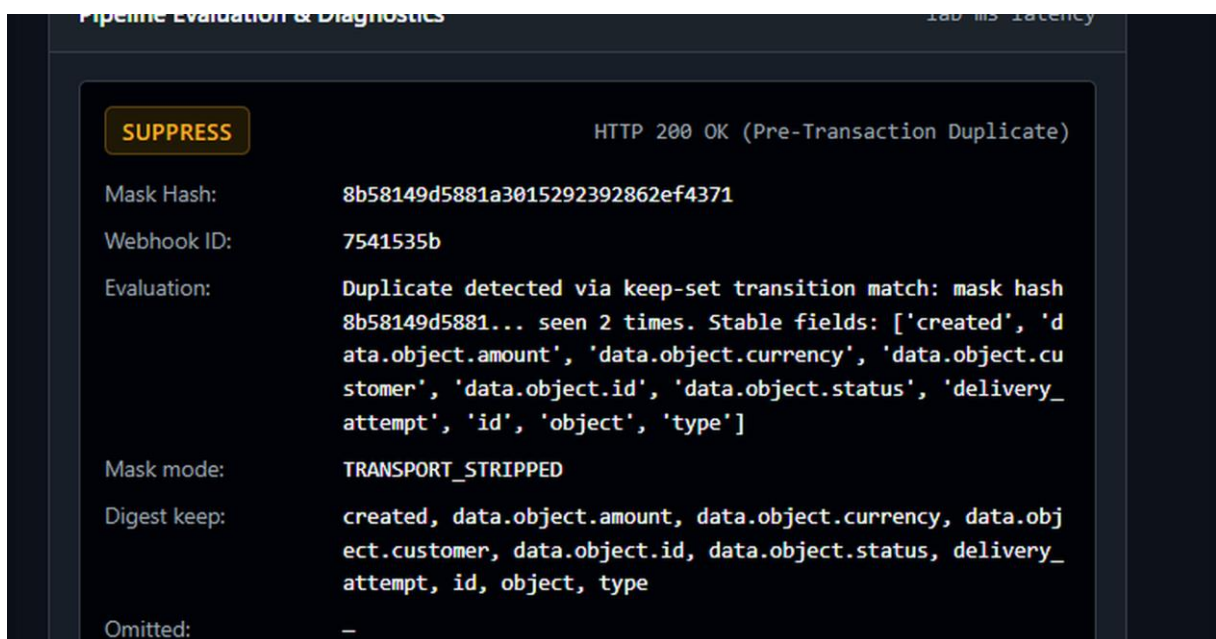


Figure 7(b). Retry decision: SUPPRESS, HTTP 200, keep-set transition match.

Live Request Log				
Timestamp	Decision	Provider	Tenant	Mask Hash
12:45:20 AM	SUPPRESS	stripe	org_production_us	8b58149d5881...
12:45:20 AM	FORWARD	stripe	org_production_us	34cbc676dd0b...

Figure 7(c). Live request log of both deliveries.

8.3 Test scenarios (store opened versus not opened)

Each row ends on the independent-claim effect. HTTP 201 / 200 apply to the FastAPI mapping in §8.2;

©VIT IPR&TTCELL
engine-level claim tests return FORWARD / SUPPRESS.

#	Scenario	Gate (laboratory)	Store	Technical point
1	First distinct identity (cold start)	FORWARD	Opened	UNCERTAIN stays in SELECTIVE; digest is new
2	Same identity after learning; only ISO timestamp (and attempt) changed	SUPPRESS	Not opened	Transport omitted without a named timestamp list
3	Distinct second identity	FORWARD	Opened	False-SUPPRESS guard
4	Opaque high-VR delivery string with low-magnitude identity (TRANSPORT_STRIPPED)	Retry SUPPRESS when VR-gated omit is on ; FORWARD when off	Not opened / opened	Remainder-safe omit isolated in ablation / claim tests
5	Unlearned wrapper key added on a retry	SUPPRESS when first-seen probation is on	Not opened	First-seen leaf omitted only while a learned path remains
6	Seeded cold tenant; identity path seeded VOLATILE	Retry SUPPRESS; distinct id FORWARD	Not opened / opened	Statistics-only seed; immunity is not inherited
7	Cold first Stripe-like retry (evt id / created / added delivery_attempt)	SUPPRESS; distinct nested payment-intent id FORWARD even if a customer string is shared	Not opened / opened	Nested keep-set transition match

Rows 1–3 are the ordinary write-barrier. Rows 4–7 isolate mechanisms that a named include/exclude list does not provide. The table does not present 0% FNR on every JSON shape.

8.4 Aggregate metrics

Evidence	Result	What it shows
py -m pytest tests/ -q	94 passed (16 September 2026)	Enablement of the recited modules
Schema-blindness AST check	SUCCESS	No field-name dictionary
standalone_verification_suite.py	6/6 pass	Zero-dependency FORWARD / SUPPRESS on Stripe-like, Shopify-like, seed, replay, and unfamiliar field names
Ablation, 40-order mix (experiments/ablation.py)	FPR 0.0%, FNR 2.5% (1 of 40; integer order_number missed on the first retry; same across all seven knob variants)	Representative laboratory mix. The 2.5% is the disclosed flat-identity / nested-keep-set class
Adaptive vs static, after drift	Adaptive 22% missed duplicates; static 100% ; both 0% false suppression	Learned mask versus a frozen exclude set after shape drift
Drift reconvergence (100 trials)	Stable→volatile median 18 ; volatile→stable median about 24.5	Bounded-window reconvergence; not a production service-level figure
Adversarial stress, 1000 events	FPR 0% in the restored run; remaining FNR is the SELECTIVE keep-set / retry-counter class	Robustness check only; not a product accuracy figure

The designed-mix result of **0% FPR (FNR 2.5%)** is the representative laboratory figure. Stress accuracy is not product accuracy. Thresholds 0.05, 0.70 and 50.0 were held fixed for every run in this table.

8.5 Unit and claim-test coverage

Item	Laboratory record
Total tests	94 , all passing
Test files	tests/test_all_modules.py; tests/test_claim_correctness.py (33 claim regressions); tests/test_concurrency.py; tests/test_schema_blindness.py; tests/test_schema_drift.py
Store coupling	test_downstream_sink_writes_only_on_forward_and_replay — SUPPRESS never writes; FORWARD and false-positive replay do

Behaviours covered (each is a FORWARD / SUPPRESS or store-write check, not a field-name guess): tenant isolation; UUID not treated as a nonce; constant-only mask does not collide distinct events; nested-array replay; seed cannot grant immunity; probation on and off; VR-gated omission on and off; SELECTIVE omission of an UNCERTAIN counter when an identity remainder exists; first Stripe-like retry suppressed while a distinct payment-intent is forwarded even when a customer string is shared; short-id events with a shared amount forwarded; DecisionRecord keep/omit after Pass B.

8.6 Limitations of the validation

The results come from a controlled laboratory mix on in-memory FastAPI with designed webhook shapes. They show that the skeleton, the learned mask and the write-barrier are architecturally correct on that mix — designed-mix FPR **0%** on the 40-order ablation, FNR **2.5%** because an integer identity may be missed on the first retry and keep-set re-applies **nested** digest keep-paths only. They are **not** a production service-level figure and do not reflect live multi-tenant traffic. Validation on multiple nodes with a persistent store is future work at **TRL 5** and is not this disclosure.

Further disclosed holes, already enabled in the Description:

- Nested keep-paths that are only shared constants can still false-SUPPRESS on some JSON shapes.
- Where first-seen probation is enabled, two distinct events whose only difference is a first-ever leaf on a new shape can collide. Difference-isolated replay then re-admits the event; that is the designed correction path, and the gate keeps running.

This section does not present 0% FNR on every JSON shape, and does not treat the adversarial stress run as product accuracy.

9. What aspect(s) of the invention need(s) protection?

The aspects that need protection are the **combination** below as an ingest **write-barrier**: the application store is opened, or it is not. The novelty is not a hash, not a named field list, and not a business rule that “duplicates must not be charged twice.” Webhook retry handling is the field of use.

Outside the scope of protection sought: field-name dictionaries; treating UUIDs or hex strings as automatic nonces; secondary-key / 2^k hedging; a distributed / Redis store (TRL-5). Those are not the laboratory embodiment. Numerical thresholds 0.05 / 0.70 / 50.0 are design defaults of that embodiment, not limitations of the recitations below.

Independent 1 (System)

A computer-implemented HTTP ingest gating system, comprising a processor coupled to a memory, the processor being operatively coupled to an application store, the memory storing processor-executable instructions that, when executed, cause the system to:

- (a) receive an HTTP JSON payload together with a tenant identifier and an optional provider identifier;
- (b) compute a value-independent skeleton of the path topology of the payload, values not being hashed into the skeleton, array indices being normalised to a common placeholder so that payloads of equivalent structure share a skeleton regardless of field values and regardless of field-name semantics;

- (c) pair same-skeleton payloads of that tenant within a bounded time window and update, for each shared path, a volatility ratio over a bounded observation window, comparing only the **intersection** of paths so that a change in array length does not create observations for missing leaves;
- (d) classify each path using a first frequency tier that assigns STABLE, VOLATILE or UNCERTAIN from the volatility ratio and a minimum observation count, UNCERTAIN remaining in the ingest digest until classified VOLATILE, and a second behavioural tier based on clock correlation, monotonic or counter behaviour and value magnitude, the two tiers being merged **bidirectionally** so that either tier may correct the other, a path recorded as protected identity being forced STABLE;
- (e) construct an ingest digest, without a human-authored include/exclude field list and without a provider event-id table, by hashing learned non-VOLATILE paths, omitting the highest raw-volatility leaves **only while a discriminating remainder remains**, the omit set being an **output** of the classification on the skeleton rather than an input list, or, where the learned mask is empty or constant-only, hashing a transport-stripped fallback that still retains a discriminator;
- (f) query a **tenant-scoped** suppression store keyed by that digest and by the tenant identifier, with no cross-tenant key fallback; and
- (g) operate a gate, operatively coupled to the application store, that **permits a store write on FORWARD** and **prevents a store write on SUPPRESS**, thereby preventing the corresponding HTTP delivery from executing a store write.

Independent 2 (Method)

A method of operating an HTTP ingest write-barrier, the method comprising:

- (a) receiving, by a processor operatively coupled to an application store, an HTTP JSON payload and a tenant identifier;
- (b) computing, by the processor, a value-independent structural skeleton of the payload, without consulting a field-name list;
- (c) updating, by the processor, bounded-window volatility records for the intersecting paths on that skeleton for that tenant, and classifying the paths by a bidirectional two-tier merge;
- (d) constructing, by the processor, a tenant-scoped ingest digest from learned non-VOLATILE paths, omitting highest raw-volatility leaves only while a discriminating remainder remains, the omit set not being supplied as a field list, or from a remainder-safe transport-stripped fallback;
- (e) evaluating a tenant-scoped gate keyed by that digest; and
- (f) **on FORWARD, opening the application store for a write; on SUPPRESS, not opening the application store** and optionally placing the payload in a time-bounded shadow buffer, thereby isolating the application store from that HTTP delivery.

The product of both independents is whether the application store is opened. Volatility ratios and magnitude splits remain control parameters of that gate. The combination is what is protected:

Recitation	Prior-art reply	Technical effect (store)
1(b)–(c), 2(b)–(c)	Not a provider schema and not a named identity field	Same-shape deliveries can be paired so a digest can be learned
1(e), 2(d)	Not Hookdeck exclude_fields and not a whole-body hash	The omit set is learned ; omit proceeds only while a discriminator remains, so a retry can SUPPRESS without colliding a distinct event
Dep. 3–4, 9, 11	Not “share the idempotency key” and not copied immunity	Cold start may use statistics only; a donor cannot unlock another tenant’s protect set; replay may re-admit a write without unlocking the mask
1(g), 2(f)	Not a business method and not a programme per se	FORWARD may write; SUPPRESS does not; /stats.downstream counts writes versus writes_avoided

No single letter (a)–(g) is the invention. A named include/exclude list plus a TTL does not produce 1(e).

Dependent 3

The system of Independent 1, wherein classification statistics may be pooled across tenants that share a

provider identifier and a skeleton, raw payload values are not pooled, and a donor tenant cannot cause the protect set of a receiving tenant to record immunity.

Dependent 4

The system of Independent 1, wherein an append-only protect set is writable **only** by difference-isolated replay of a suppressed payload against a forwarded payload, and is re-applied on every volatility-record read, so that later drift and pooled seed data cannot unlock a protected path.

Dependent 5

The system of Independent 1, wherein omission of highest raw-volatility leaves from a SELECTIVE digest proceeds only while the remainder is mixed, is not high-volatility, and is not a low-magnitude numeric, and wherein omission from a collapsed (transport-stripped) digest proceeds only while any discriminator remains.

Dependent 6

The system of Independent 1, wherein a leaf that has no volatility record on the current skeleton and is absent from the known-path base of the tenant is provisionally omitted from the fallback digest only while at least one previously learned leaf remains, a first-ever shape keeping a strict digest so that two distinct events do not collide on an empty remainder.

Dependent 7

The system of Independent 1, wherein, on a digest miss, the same gate re-applies this delivery's **nested** digest keep-paths to a previously admitted payload of the same tenant that is path-set compatible (subset or superset), and issues SUPPRESS if those nested keep values match and are not value-shape transport, top-level keep-paths that caused the miss not being re-applied, and value magnitude not being used as an identity test.

Dependent 8

The system of Independent 1, wherein UUIDs and hexadecimal tokens are not treated as automatic nonces solely by token shape, and remain eligible as digest identity unless classified VOLATILE by change frequency or by the behavioural tier.

Dependent 9

The method of Independent 2, wherein, on a false-positive report, divergent non-transport paths are isolated and appended to the protect set, the held payload may be re-admitted as a store write, and the remainder of the learned mask is not unlocked.

Dependent 10

The system of Independent 1, wherein the skeleton hash uses array indices normalised to a common placeholder so that a change in array length does not split the pairing group, and wherein pairing, the volatility ratio, and the ingest digest still use indexed paths, updates remaining restricted to the path intersection.

Dependent 11

The method of Independent 2, wherein classification statistics may be pooled across tenants that share a provider identifier and a skeleton, raw values are not pooled, and pooled statistics cannot write the protect set of the receiving tenant.

Dependent 12

The method of Independent 2, wherein, on a digest miss, nested digest keep-paths of this delivery may be re-applied to a previously admitted payload of the same tenant, SUPPRESS issuing only if those nested values match and are not value-shape transport.

 VIT Vellore Institute of Technology (Deemed to be University under section 3 of UGC Act, 1956)	Document No.	02-IPR-R003
	Issue No/Date	1/14.09.2026
	Amd. No/Date	0/00.00.0000

Invention Disclosure Format (IDF)-B**10. What is Technology readiness level of your invention? (Tick the appropriate TRL)**

Research			Development			Deployment		
TRL 1	TRL 2	TRL 3	TRL 4	TRL 5	TRL 6	TRL 7	TRL 8	TRL 9
Basic Principles observed	Technology concept formulated	Experimental proof of concept	Technology validated in a lab	Technology validated in a relevant environment (industrially relevant in case of key enabling technologies)	Technology demonstrated in a relevant environment (industrially relevant in case of key enabling technologies)	System prototype demonstration in an operational environment	System complete and qualified	Actual system proven in an operational environment (competitive manufacturing in case of key enabling technologies, or in space)
			☑					

Justification: The invention has been validated in a laboratory environment (**TRL 4**). Ten cooperating modules sit in front of an application store and are exercised through an HTTP laboratory server. Ninety-four automated tests pass, and a schema-blindness check confirms that the gate consults no field-name dictionary. On a designed laboratory mix, false-positive suppression is 0.0% and false-negative admission is 2.5%. After payload-shape drift, a learned mask misses 22% of duplicates, against 100% for a frozen static exclude, both with 0% false suppression. Figure 7 records the same write-barrier on live HTTP: FORWARD opens the store (HTTP 201); SUPPRESS does not (HTTP 200). Ingest state remains in-memory on a single node. Validation in a relevant (persistent or multi-node) environment would be TRL 5 and is outside this disclosure.

-----END OF THE DOCUMENT-----